Your Own Packages and Interfaces

Creating Packages, Defining Interfaces, Implementing Services, Parameters and Actions

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 3, Lecture B

Session Structure

You type. We stop at every checkpoint until the room is together.

1. Custom Interface Packages
2. Publishing on a Topic
3. Service Server and Client
4. Executor Deadlock
5. Parameters
6. Action Clients

Your Week 2 workspace, one new interfaces package, one new nodes package.

Outline

Part 1

Custom Interface Packages

Two Tools: `ament` and `colcon`

A workspace is a building site. **ament** is the standard each package is built to; **colcon** is the foreman — reads the plans, decides who works first, and never picks up a tool.

	ament	**colcon**
What	The build system of *one* package	The tool that builds *all* of them
Scope	One package	The whole workspace
You	Choose it: `--build-type`	Run it: `colcon build`

Two flavours of ament, and this week you need both: `ament_python` builds your nodes with setuptools; `ament_cmake` builds the interfaces package, because turning a `.msg` file into a class you can import is a CMake job.

colcon decides *when* each package is built. ament decides *how*.

Package Layout for Custom Interfaces

```
$ cd ~/ee414_ws/src
$ ros2 pkg create ee414_w03_interfaces --build-type ament_cmake
$ ros2 pkg create ee414_w03_nodes --build-type ament_python \
      --dependencies rclpy std_msgs
```

Note the different build types. `ament_cmake` for the definitions, because interface generation is a CMake job; `ament_python` for your nodes, as last week.

Under the hood

Trying to put a `.msg` in your Python package is the first thing most people try. It builds, generates nothing, and the import fails at run time with a message that does not mention the cause.

Package Creation Output

```
going to create a new package
package name: ee414_w03_interfaces
build type: ament_cmake
creating folder ./ee414_w03_interfaces
creating ./ee414_w03_interfaces/package.xml
creating source and include folder
creating folder ./ee414_w03_interfaces/src
creating folder ./ee414_w03_interfaces/include/ee414_w03_interfaces

$ ros2 pkg create ee414_w03_nodes --build-type ament_python --dependencies rclpy std_msgs
going to create a new package
package name: ee414_w03_nodes
build type: ament_python
dependencies: ['rclpy', 'std_msgs']
creating folder ./ee414_w03_nodes
creating ./ee414_w03_nodes/package.xml
creating source folder
creating folder ./ee414_w03_nodes/ee414_w03_nodes
creating ./ee414_w03_nodes/setup.py
```

Read the echoed `build type` and `dependencies` lines back before you carry on. That is the whole check.

A Common `pkg create` Error

Put the package name *after* `--dependencies` and it disappears:

```
$ ros2 pkg create --build-type ament_python \
      --dependencies rclpy std_msgs ee414_w03_nodes
```

```
                    [--maintainer-name MAINTAINER_NAME]
                    [--node-name NODE_NAME] [--library-name LIBRARY_NAME]
                    package_name
ros2 pkg create: error: the following arguments are required: package_name
```

`--dependencies` takes a *list*, so it swallows `ee414_w03_nodes` as a third dependency and then complains that no package name was given. The error names `package_name` — the one thing you did type.

Package name first. Options after.

Writing the Interface Definitions

ee414_w03_interfaces/msg/WheelState.msg

```
std_msgs/Header  header
float64          left_rad_s
float64          right_rad_s
bool             estopped
```

ee414_w03_interfaces/srv/ResetOdom.srv

```
bool zero_heading
---
bool success
string message
```

Folders msg/ and srv/, exactly. Files CamelCase, fields snake_case — the generator enforces both.

Build Configuration in CMakeLists.txt

```
find_package(rosidl_default_generators REQUIRED)
find_package(std_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/WheelState.msg"
  "srv/ResetOdom.srv"
  DEPENDENCIES std_msgs
)
```

And two dependencies in `package.xml`. The exact lines are in the lab sheet — **copy them**. Nobody memorises this, including me.

`DEPENDENCIES std_msgs` is required because `WheelState` embeds a `Header`. Omit it and the build fails with a message about an unknown type.

Building and Verifying the Package

```
$ cd ~/ee414_ws
$ colcon build --symlink-install
$ source install/setup.bash
```

```
$ colcon build --symlink-install
Starting >>> ee414_w03_interfaces
Finished <<< ee414_w03_interfaces [6.09s]
Starting >>> ee414_w03_nodes
Finished <<< ee414_w03_nodes [1.23s]

Summary: 2 packages finished [7.48s]
```

`ee414_w03_interfaces` must finish *before* `ee414_w03_nodes` starts — colcon works that out from `package.xml`. If it builds them the other way round, your dependency line is missing.

Generated Interfaces: `ros2 interface show`

```
$ ros2 interface show ee414_w03_interfaces/msg/WheelState
std_msgs/Header  header
        builtin_interfaces/Time stamp
                int32 sec
                uint32 nanosec
        string frame_id
float64          left_rad_s
float64          right_rad_s
bool             estopped
$ ros2 interface show ee414_w03_interfaces/srv/ResetOdom
bool zero_heading
---
bool success
string message
```

`Header` arrives **expanded** — `show` follows nested types down to their primitives. Your `.msg` had four lines; this has eight.

Checkpoint 1

You should see	If you do not
Your four fields, with `Header` expanded into `stamp` and `frame_id`	The generator ran. Carry on.
`Unknown interface`	The build did not generate it. Did you `source install/setup.bash` *after* the build?
A build error naming `Header`	`DEPENDENCIES std_msgs` is missing from `rosidl_generate_interfaces`.

Checkpoint 1. Everyone can `interface show` their own message.

Fix this before writing a single node. Everything after this slide assumes the generated Python class exists.

Outline

Part 2

Publishing on a Topic

Using a Custom Message in a Node

```python
from ee414_w03_interfaces.msg import WheelState
msg = WheelState()
msg.header.stamp = self.get_clock().now().to_msg()
msg.header.frame_id = 'base_link'
msg.left_rad_s  = 2.4
msg.right_rad_s = 2.4
msg.estopped = False
self.pub.publish(msg)
```

Import path is `package.msg`, then the type name. Fields are plain attributes.

Stamping is a habit, not a formality

`self.get_clock().now()` — never `time.time()`. In simulation the clock is not wall time, and a node using it disagrees with the whole robot. Week 8 shows this.

Verifying the Custom Message on the Wire

```
$ ros2 topic echo /wheel_state --once
header:
  stamp:
    sec: 1788611340
    nanosec: 637558705
  frame_id: base_link
left_rad_s: 2.4
right_rad_s: 2.4
estopped: false
---
```

Your own type, inspectable from the command line with no extra work — because you declared it properly rather than packing numbers into a string. The `---` at the bottom is the message separator, not part of your data.

Checkpoint 2. Everyone sees their own fields in `echo`.

Outline

Part 3

Service Server and Client

The Service Server

```python
from ee414_w03_interfaces.srv import ResetOdom

class OdomNode(Node):
    def __init__(self):
        super().__init__('odom_node')
        self.x = 12.5
        self.create_service(ResetOdom, 'reset_odom', self.on_reset)

    def on_reset(self, request, response):
        self.x = 0.0
        response.success = True
        response.message = 'odometry zeroed'
        return response
```

A service callback receives `request`, fills `response`, and **returns it**. Forgetting the return is the most common mistake — the caller then waits forever.

Testing the Service from the Command Line

```
$ ros2 service list | grep reset
/reset
/reset_odom
$ ros2 service call /reset_odom ee414_w03_interfaces/srv/ResetOdom "{zero_heading: true}"
waiting for service to become available...
requester: making request: ee414_w03_interfaces.srv.ResetOdom_Request(zero_heading=True)

response:
ee414_w03_interfaces.srv.ResetOdom_Response(success=True, message='odometry zeroed')
```

One half at a time. The server is now proven. Anything that goes wrong from here is in the client — which is a much smaller place to look.

Checkpoint 3. Everyone has called their own service from the terminal.

Outline

Part 4

Executor Deadlock

Attempt 1: Blocking on the Future

Add this to a *subscriber callback*:

```python
def on_msg(self, msg):
    future = self.cli.call_async(ResetOdom.Request())
    rclpy.spin_until_future_complete(self, future)
    self.get_logger().info('never printed')
```

```
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  File "/opt/ros/jazzy/lib/python3.12/site-packages/rclpy/__init__.py", line 274, in spin_until
_future_complete
    executor.spin_until_future_complete(future, timeout_sec)
  File "/opt/ros/jazzy/lib/python3.12/site-packages/rclpy/executors.py", line 370, in spin_unti
l_future_complete
    self._enter_spin()
  File "/opt/ros/jazzy/lib/python3.12/site-packages/rclpy/executors.py", line 222, in _enter_sp
in
    raise RuntimeError('Executor is already spinning')
RuntimeError: Executor is already spinning
[ros2run]: Process exited with failure 1
```

`RuntimeError: Executor is already spinning`. ROS 2 Jazzy refuses rather than freezing — you asked `spin` to start while it was already running. **This is the kind version of the mistake**: there is a name to search for.

Attempt 2: Synchronous `call()`

The obvious repair. `call()` instead of `call_async()` and no spinning:

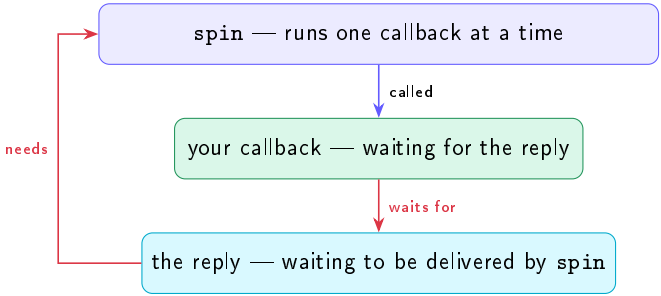
```python
def on_msg(self, msg):
    response = self.cli.call(ResetOdom.Request())
    self.get_logger().info(f'never printed: {response.message}')
```

```
$ ros2 run ee414_w03_nodes reset_client --ros-args -p mode:=hang
[INFO] [1788611874.555821183] [reset_client]: waiting for /reset_odom ...
[INFO] [1788611874.831060286] [reset_client]: mode=hang - subscribed to /wheel_state
[INFO] [1788611875.076002188] [reset_client]: callback: calling the service, waiting for the re
ply ...
```

Three lines, then nothing. No error, no traceback, no exit — the process is still alive and will sit there until you kill it. **This is the dangerous one**, and it is what *deadlock* means.

Checkpoint 4. Everyone's node is now hung. Look at it for a moment.

The Cause of the Deadlock



A circle. Your callback is waiting for something only `spin` can deliver, and `spin` is inside your callback.

The Fix: Callback-Based Completion

```python
def on_msg(self, msg):
    future = self.cli.call_async(ResetOdom.Request())
    future.add_done_callback(self.on_reply)
    return

def on_reply(self, future):
    self.get_logger().info(f'reply: {future.result().message}')
```

The callback **returns immediately**. `spin` regains control, delivers the reply, and calls `on_reply`. Two short callbacks instead of one that waits.

In the field

The rule: a callback never waits. It does its work and returns; anything that takes time becomes a second callback. Internalise this and a whole class of ROS 2 hang stops happening to you.

The Corrected Client in Operation

```
[INFO] [1788611898.076364061] [reset_client]: reply: odometry zeroed
[INFO] [1788611898.226073291] [reset_client]: reply: odometry zeroed
[INFO] [1788611898.321366203] [reset_client]: reply: odometry zeroed
[INFO] [1788611898.576233510] [reset_client]: reply: odometry zeroed
[INFO] [1788611898.725191193] [reset_client]: reply: odometry zeroed
[INFO] [1788611898.821207519] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.076258072] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.225845153] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.321166643] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.576497619] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.725901031] [reset_client]: reply: odometry zeroed
[INFO] [1788611899.821533100] [reset_client]: reply: odometry zeroed
[INFO] [1788611900.077095942] [reset_client]: reply: odometry zeroed
[INFO] [1788611900.225697574] [reset_client]: reply: odometry zeroed
```

The same node, the same service, the same subscriber callback — replying at the rate the publisher publishes. Nothing was made faster; the waiting was moved.

Outline

Part 5

Parameters

Declaring and Reading Parameters

```python
class Driver(Node):
    def __init__(self):
        super().__init__('driver')
        self.declare_parameter('max_speed', 0.22)
        self.declare_parameter('frame_id', 'base_link')
        self.create_timer(0.1, self.tick)

    def tick(self):
        v = self.get_parameter('max_speed').value
        self.get_logger().info(f'v = {v}')
```

Read the parameter **inside** `tick`, not once in `__init__`. That is what makes a live change take effect.

Modifying a Parameter at Runtime

```
$ ros2 param list /driver
  frame_id
  max_speed
  start_type_description_service
  use_sim_time
$ ros2 param get /driver max_speed
Double value is: 0.22
$ ros2 param set /driver max_speed 0.05
Set parameter successful
$ ros2 param get /driver max_speed
Double value is: 0.05
```

Watch the node's output change without restarting it. **That is tuning**, and it is how you will find your controller gains in Week 5 and your costmap sizes in Week 11.

Checkpoint 5. Everyone has changed a running node's behaviour from the terminal.

Persisting Parameter Values

```
Node(package='ee414_w03_nodes', executable='driver',
     parameters=[{'max_speed': 0.15, 'frame_id': 'base_link'}])
```

A value set from the terminal is gone when the node restarts. A value in a launch file is **reproducible** — and reproducibility is what a demo day needs.

The workflow

Tune live with `param set` until it behaves. Then write the value you found into the launch file. Students who skip the second step spend demo day wondering why the robot is worse than it was yesterday.

Outline

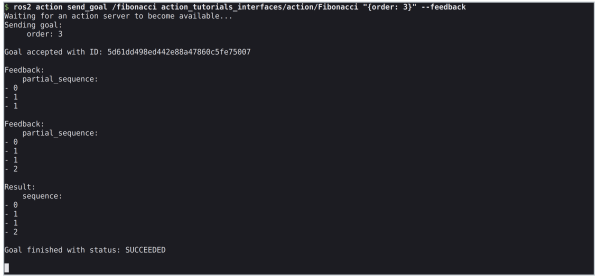
Part 6

Action Clients

Observing an Action from the Command Line

In one terminal, start a server. In another, ask it for a short sequence:

```
$ ros2 run action_tutorials_py fibonacci_action_server
```



Run it **without** `--feedback` as well, and compare. The difference between the two runs is the entire reason actions exist.

The Action Client

```python
self.ac = ActionClient(self, Fibonacci, 'fibonacci')
self.ac.wait_for_server()

goal = Fibonacci.Goal()
goal.order = 10
future = self.ac.send_goal_async(goal, feedback_callback=self.on_fb)
future.add_done_callback(self.on_accepted)

def on_fb(self, fb):
    self.get_logger().info(f'progress: {fb.feedback.partial_sequence}')
```

`send_goal_async`, a future, a done callback — the same shape as the service client. One extra piece: a feedback callback, fired repeatedly while the server works.

Checkpoint 6. Everyone has seen feedback arrive from a running goal.

Summary of Capabilities

You built	Which means
Your own message type	Data travels as named typed fields, inspectable by tools you did not write
A service, both halves	A command you can confirm was received and succeeded
An async client	Callbacks that never block — the habit that prevents the hang
Parameters	Behaviour you can tune without editing or rebuilding
An action client	Long jobs you can watch and cancel

Together with Week 2, that is the whole of ROS 2's communication surface. **Everything from here is robotics.**

Deliverables and Next Session

Leaving this room: an interfaces package that builds, a custom message published and echoed, a service called from both the terminal and a client, a deadlock you caused and fixed, a parameter changed live, and feedback from a running action.

Assignment 1 is due at the end of this week. Commit and push it.

Next week the robot moves

Week 4 spends its theory hour on differential-drive kinematics, and its second hour driving a TurtleBot3 in Gazebo with `/cmd_vel`. **Make sure Gazebo launches on your machine before then** — the setup guide has the command. Finding out in the session that it does not is an expensive way to spend ninety minutes.